CORRIGÉ · PROBLÈME · ANNALES MP

Codage arithmétique

Énoncé

Consignes Python : pas de fonctions internes sur les listes ou chaînes (`min`, `max`, `count`, `remove`…) ; seules `len`, `append`, le tranchage et la concaténation sont permis. Programmes commentés si nécessaire.

Codage arithmétique : fréquences a : 0,2, b : 0,1, c : 0,2, d : 0,4, e : 0,1, d'où les intervalles a : $[0;0{,}2[$, b : $[0{,}2;0{,}3[$, c : $[0{,}3;0{,}5[$, d : $[0{,}5;0{,}9[$, e : $[0{,}9;1[$. On part de $[0;1[$ et, à chaque caractère, on garde la portion correspondante du sous-intervalle courant. Exemple : `'dac'` donne $[0{,}5;0{,}9[$, puis $[0{,}50;0{,}58[$, puis $[0{,}524;0{,}540[$. `codeCar(car, g, d)` renvoie le sous-intervalle de $[g,d[$ associé à `car`.

1. Intervalle correspondant à `'bac'`.
2. Écrire `codage(s:str) -> (float, float)` qui renvoie l'intervalle final (g,d).

Décodage : on repère dans quelle succession d'intervalles se trouve x. Pour $x = 0{,}123$: `'a'`, puis `'d'` (sous-intervalle $[0{,}10;0{,}18[$), etc.

1. Caractère qui suit `'ad'` pour $x = 0{,}123$, avec le sous-intervalle utilisé.
2. Donner deux chaînes correspondant à 0,2. D'où vient l'ambiguïté ?

On ajoute un caractère de fin `'#'`, de plage non vide près de 0. `decodeCar(x, g, d)` renvoie le caractère dont la plage dans $[g,d[$ contient x.

1. Écrire `decodage(x:float) -> str` (caractère `'#'` compris).

Décoder l'énoncé

Ce que l'énoncé te donne.

- Le sous-intervalle d'un caractère dans $[g,d[$ est $[g+a(d-g),\ g+b(d-g)[$ si sa plage dans $[0,1[$ est $[a,b[$.

Ce qu'on te demande. Coder une chaîne par un intervalle, puis décoder un réel.

Corrigé

Question 10

Étape 1. Q10. Intervalle de « bac »

Règle : si la plage du caractère dans $[0;1[$ est $[a;b[$, le nouvel intervalle est $[g+a(d-g)\,;\ g+b(d-g)[$.

- `'b'` (plage $[0{,}2;0{,}3[$) dans $[0;1[$: $[0{,}2;0{,}3[$, de longueur $0{,}1$.
- `'a'` (plage $[0;0{,}2[$) dans $[0{,}2;0{,}3[$: $[0{,}2+0\times0{,}1\,;\ 0{,}2+0{,}2\times0{,}1[=[0{,}2;0{,}22[$, de longueur $0{,}02$.
- `'c'` (plage $[0{,}3;0{,}5[$) dans $[0{,}2;0{,}22[$: $[0{,}2+0{,}3\times0{,}02\,;\ 0{,}2+0{,}5\times0{,}02[$.

$$\texttt{'bac'} \longmapsto [0{,}206\,;\ 0{,}21[.$$

Vérification : la longueur finale vaut $0{,}1\times0{,}2\times0{,}2=0{,}004$, produit des fréquences.

Question 11

Étape 1. Q11. La fonction codage

```
def codage(s):
    g, d = 0, 1                    # intervalle de départ [0, 1[
    for i in range(len(s)):
        g, d = codeCar(s[i], g, d)  # on affine avec le caractère lu
    return (g, d)
```

Exemple : `codage('dac')` renvoie $(0{,}524\,;\ 0{,}540)$.

Question 12

Étape 1. Q12. Le caractère après « ad »

Après `'ad'`, l'intervalle courant est $[0{,}10;0{,}18[$, de longueur $0{,}08$. La position relative de x vaut

$$\frac{x-0{,}10}{0{,}08}=\frac{0{,}023}{0{,}08}\approx 0{,}29\in[0{,}2;0{,}3[,$$

plage de `'b'`. Le sous-intervalle correspondant est

$$[0{,}10+0{,}2\times0{,}08\,;\ 0{,}10+0{,}3\times0{,}08[=[0{,}116\,;\ 0{,}124[\ni 0{,}123$$: le caractère suivant est `'b'`.

Question 13

Étape 1. Q13. Une ambiguïté

$0{,}2 \in [0{,}2; 0{,}3[$: 0,2 code `'b'`. Mais 0,2 est aussi la borne gauche de l'intervalle de `'ba'`, $[0{,}2; 0{,}22[$, et de celui de `'baa'`, $[0{,}2; 0{,}202[$, etc.

> `'b'` et `'ba'` (et aussi `'baa'`, `'baaa'`…) correspondent toutes à 0,2.

Origine : le réel ne dit pas où la chaîne s'arrête. Comme la plage de `'a'` commence à 0, ajouter des `'a'` à la fin garde la même borne gauche : on ne sait pas quand arrêter le décodage.

Question 14

Étape 1. Q14. La fonction decodage

```
def decodage(x):
    s = ''
    g, d = 0, 1
    c = ''
    while c != '#':                # '#' marque la fin de la chaîne
        c = decodeCar(x, g, d)     # caractère dont la plage contient x
        s = s + c
        g, d = codeCar(c, g, d)    # on descend dans son sous-intervalle
    return s
```

La boucle s'arrête au premier `'#'` lu, qui est inclus dans la chaîne renvoyée, comme demandé. (Avec des flottants de précision limitée, cela ne marche que pour des chaînes courtes, comme le rappelle l'énoncé.)

Le piège

En Q10, multiplier les bornes au lieu de multiplier la longueur : le nouvel intervalle est décalé de g et mis à l'échelle de $d - g$.

La méthode

Intervalle emboîté : $[g, d[\to [g + a(d-g), g + b(d-g)[$. Pour décoder, on refait le même chemin en cherchant à chaque étape la plage qui contient x.